

tool

Anonymous Author(s)*

ABSTRACT

Code translation—converting code from one programming language to another while preserving functionality—is critical for legacy system migration, cross-platform development, and ecosystem modernization. Traditional rule-based or manual approaches are limited in scope and efficiency. While large language models (LLMs) have shown promise in code generation, most existing research focuses on translating isolated methods or small files, leaving project-level translation—where accumulated language paradigm differences become significant—largely unexplored.

This paper proposes TOOL, a hierarchical, macro-to-micro translation framework designed to mitigate error propagation in large-scale code translation. The framework first performs static analysis to extract call graphs and decompose code into functional modules using a custom Java parser tool. Translation then proceeds in two layers: (1) at the class level, where overall architecture and header files are designed with language-specific pattern mapping, and (2) at the method level, where implementations are translated bottom-up along call chains to ensure dependency stability. To bridge library and runtime differences, the framework dynamically generates necessary dependencies and an autonomous agent iteratively compiles, tests, and repairs the translated code using intelligent error analysis and fix strategies.

We evaluate TOOL by translating several cohesive Java modules to C++, two languages with significant differences in underlying execution models. Experimental results demonstrate that ... **TODO: Add experimental results**

Keywords: Code translation, Large language models, Project-level translation, Java, C++, Static analysis, Hierarchical translation

1 INTRODUCTION

The increasing need for software migration, cross-language interoperability, and multi-platform deployment has made automated code translation a valuable research area. Early techniques relied on handcrafted rules or template-based transformations, which are difficult to scale. Recent advances in LLMs have enabled more flexible and context-aware code generation, yet their application in translation remains largely confined to small, self-contained code snippets. Translating entire projects or interconnected modules introduces challenges such as architectural mismatch, dependency management, and compounding semantic gaps between languages—especially those with different runtime paradigms like Java (managed, JVM-based) and C++ (unmanaged, native).

This paper addresses the problem of error accumulation in project-level translation. We argue that translating files or methods independently without a high-level understanding of the module’s purpose leads to irreconcilable inconsistencies. Instead, we propose a top-down understanding followed

by bottom-up translation, where the LLM first comprehends the macro-structure of a functional module, plans the overall translation strategy, and then executes translation in a layered manner.

Our main contributions are:

- **A hierarchical translation framework (tool)** that separates class-level design from method-level implementation, enabling macro-to-micro translation with reduced error propagation.
- **A call-graph-guided, bottom-up translation order** that stabilizes dependencies by translating leaf methods first, ensuring callees are available when translating callers.
- **An autonomous compilation and repair agent** that validates and corrects the translated code using intelligent error analysis, supporting iterative compilation with automated fix strategies.

2 RELATED WORK

TODO: add references

Rule-Based and IR-Based Translation. Early work used abstract syntax trees (ASTs) and intermediate representations (IRs) to transform code between syntactically similar languages. These methods require extensive manual rules and struggle with semantic disparities.

Neural Machine Translation for Code. Sequence-to-sequence models were applied to code translation, but performance was limited by training data and generalization.

LLM-Based Code Generation. Models like Codex and GPT-4 have demonstrated strong few-shot code generation capabilities. Several studies fine-tune LLMs for translation between specific language pairs, but evaluation is typically done on isolated functions or competition-style problems.

Project-Level Translation. Research on translating entire projects is nascent. Challenges include maintaining inter-file references, preserving architectural patterns, and handling external libraries. Our work extends this direction by introducing a structured, phase-based approach to contain cross-language divergence.

3 FRAMEWORK IMPLEMENTATION

This section presents the detailed implementation of TOOL, our hierarchical Java-to-C++ translation framework. The system is composed of four main components: (1) Static Analysis Tool, (2) Hierarchical Translation Engine and (3) Autonomous Compilation Agent.

Figure ?? illustrates the overall architecture of TOOL. The framework operates in a pipeline fashion, with each component processing the output of the previous one.

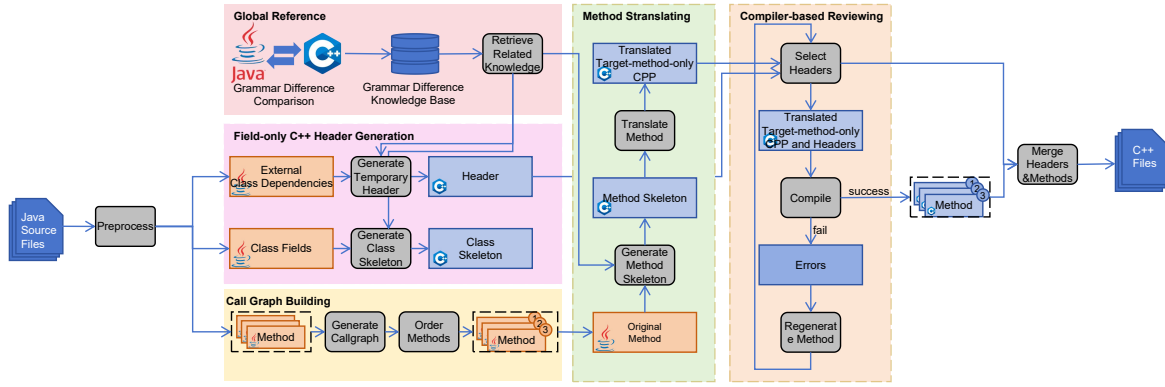


Figure 1: Overall architecture of tool showing the four main components and data flow.

3.1 Static Analysis Tool

The static analysis tool is implemented as a standalone Java application using JavaParser library. The parser performs the following operations:

- (1) **AST Parsing:** Extracts abstract syntax trees from Java source files
- (2) **Code Decomposition:** Separates class declarations from method implementations
- (3) **Dependency Extraction:** Identifies import statements and inheritance relationships
- (4) **Metadata Generation:** Creates structured representations for translation

After parsing, the project information is stored in a graph, which contains nodes representing class headers or methods and edges representing dependencies. This graph is used as the input for the translation engine.

3.2 Hierarchical Translation Engine

3.2.1 Basic process of translation.

Code splitting. Due to the fact that the scale of the project far exceeds the context limit of the large model, the code must be split in a reasonable and orderly manner to facilitate batch translation. We divide the nodes in the graph obtained in the previous section into two categories: one category records class-related information, including inheritance relationships, fields, various method declarations, etc.;

Header and Method data structures

```
class Header:
    def __init__(self, key, source_code):
        self.key = key # Class name
        self.source_code = source_code # Original Java code
        self.type = "" # class/interface/enum
        self.methods = [] # Method instances
        self.imports = [] # Import statements
        self.translated_code = "" # Generated C++ code
        self.header_files = [] # Dependencies

class Method:
    def __init__(self, key, code, source_tag, header):
        self.key = key # Class:Method(params)
        self.code = code # Java implementation
        self.children = set() # Called methods
        self.parents = set() # Calling methods
        self.translated_code = "" # C++ implementation
        self.compile_output = "" # Compilation results
```

Figure 2: Header and Method data structures

the other category records the specific implementation information of methods. Our main translation objects are these two types of code fragments. For the translation from Java to C++, the former will generate C++ header files after analysis by the large model, and the latter will generate the implementation of C++ methods.

Translation order. Among the two types of nodes mentioned above, the head node is translated first. After that, all method nodes are topologically sorted according to their call relationships and then translated in sequence. The advantage of this order is that when translating the head node, the

large model can focus on macro control and framework determination. When translating a certain method node next, the information of the head node to which the method node belongs and the information of previously translated nodes can serve as context to provide additional information. Using this method, the scale of translation can be expanded arbitrarily without worrying about the input length exceeding the model's limit, and it also reduces the occurrence of situations where the previously translated parts and the later translated parts are difficult to align. **TODO: Add a figure showing the translation order**

3.2.2 Two-Phase Strategy. While translating each node in graph(header or method), the translation engine implements a dual-phase approach:

Phase 1: Scheme Generation. In this phase, the large model understands and analyzes the code segment to be translated. For example, when translating "head node", the large model will determine the strategies for converting between language features (memory allocation, generics, standard library mapping, etc.) and type mapping. The figure ?? shows an example of a prompt for generating a header scheme.

```
<system>
You are an advanced AI code migration assistant, participating in a project that migrates
a Java repository to a C++ repository. According to the instructions, you will focus on only
one issue at a time and gradually handle tasks related to code translation, dependency management,
and compilation.
</system>

<context>
The following is a Java method from class {class_name}
```java
{java_method_code}
``` {data-source-line="10"}
Now, it is supposed to be translated to the following C++ method:
{cpp_declaration}
</context>

<instruction>
Provide a detailed translation plan for the Java method, include the following contents:
1. Mapping of data type, including those that can be directly mapped and those that require
custom implementation. mark this part with <mapping></mapping> tags;
2. The necessary refactoring to align differences in language features. Mark this part with
<refactoring></refactoring> tags;
3. Additional custom classes that will be used in C++ code and need to be declared, use //None/
if no custom classes are needed. Mark this part with <additional-class></additional-class> tags;
4. Additional custom methods that will be used in C++ code and need to be declared(including
class methods), use //None/ if no custom methods are needed. Mark this part with
<additional-method></additional-method> tags;
5. Other precautions, mark this part with <precaution></precaution> tags;

SPECIAL CASE: If this is a very short boilerplate method and does not contain complex
dependencies, then omit all the above content and directly provide the translation result, mark
this part with <implementation></implementation> tags, except for this case, only provide the
solution and do not provide specific code.
</instruction>

<note>
- Focus only on the current method, DO NOT attempt to implement the entire class.
- All the above require specific solutions to be provided, DO NOT provide ambiguous answers.
- The followings are fields of the class {class_name} for reference:{class_fields_str}
- The following classes and methods have already been implemented in C++, the header files
where these methods are located have the same name as the classes they belong to, they DO NOT
NEED to appear in <additional-class> or <additional-method> tags;\n {implemented_methods}
</note>

<suggestion>
{suggestions}
</suggestion>
```

Figure 3: header scheme generation prompt

Phase 2: Code Generation. At this stage, the generation of code in the target language officially begins, and by this time, the large model already has quite a rich context. ?? shows a example of method translation prompt, include:

- The translation scheme generated in Phase 1
- The method signature declared in the head node
- Other fields and methods related to the method

```
<system>
You are an advanced AI code migration assistant, participating in a project that migrates
a Java repository to a C++ repository. According to the instructions, you will focus on only
one issue at a time and gradually handle tasks related to code translation, dependency management,
and compilation.
</system>

<context>
The following is a Java method from class {class_name}
```java
{java_method_code}
``` {data-source-line="10"}
Now, it is supposed to be translated to the following C++ method:
{translated_cpp_declaration}
</context>

<scheme>
Translation Scheme:
{scheme}
</scheme>

<instruction>
1. Provide the translated C++ method implementation based on the above scheme, mark this
part with <implementation></implementation> tags.
2. Include the necessary header files (include the "{class_name}.h")
3. Declare the classes and methods in <additional-class> and <additional-method> tags (if
any) in a /"temp_header.h/" file. Mark this part with <dependency></dependency> tags.
</instruction>

<note>
- The header file has already been written. You just need to write the out-of-body
implementation of this function, and there's no need to repeat the class and field declarations.
- remember to include /" {class_name}.h/"
- The followings are the fields of the class {class_name} for reference:{class_fields}
- The following classes and methods have already been implemented, you can just include them
instead of re-declaring.
{external_methods_implementations_str}
</note>
```

Figure 4: method translation prompt

3.3 Autonomous Compilation Agent

3.3.1 Agent Architecture. The compilation agent operates as an autonomous LLM-powered system with tool access:

- **Tool Suite:** File operations, compilation, error analysis
- **Decision Engine:** LLM-based error diagnosis and fix planning
- **State Management:** Conversation history and progress tracking

3.3.2 Tool Interface. The agent has access to four core tools:

3.3.3 Error Resolution Strategy. The agent follows a systematic approach to error resolution:

- (1) **Compilation:** Attempt to compile the target file

Agent tool definitions

```
tools = {  
    "compile_file": compile_cpp_file,  
    "read_file": read_file_content,  
    "write_file": modify_file_content,  
    "create_file": create_new_file  
}  
  
def compile_cpp_file(file_path):  
    result = subprocess.run(  
        ["clang++", "-c", file_path, "-o", obj_file],  
        capture_output=True, text=True  
    )  
    return {  
        "success": result.returncode == 0,  
        "log": result.stdout + result.stderr  
    }
```

Figure 5: Agent tool definitions

- (2) **Analysis:** Parse error messages and identify root causes
- (3) **Planning:** Generate fix strategies using LLM reasoning
- (4) **Execution:** Apply fixes through tool operations
- (5) **Validation:** Re-compile and verify resolution

This comprehensive implementation enables robust, scalable translation of Java projects to C++ while maintaining code quality and minimizing error propagation through the hierarchical approach and intelligent compilation repair mechanisms.

REFERENCES

465		523
466		524
467		525
468		526
469		527
470		528
471		529
472		530
473		531
474		532
475		533
476		534
477		535
478		536
479		537
480		538
481		539
482		540
483		541
484		542
485		543
486		544
487		545
488		546
489		547
490		548
491		549
492		550
493		551
494		552
495		553
496		554
497		555
498		556
499		557
500		558
501		559
502		560
503		561
504		562
505		563
506		564
507		565
508		566
509		567
510		568
511		569
512		570
513		571
514		572
515		573
516		574
517		575
518		576
519		577
520		578
521		579
522		580